Four-rule iPRA with reusable clauses

An explicit presentation, exact proof translations, and a smaller complete checker

Prepared for Vladimir Reshetnikov 11 November 2026

Comparison pinned to ProveIt commit `443cff4956`.

Abstract

We give an incremental simplification of the current ConciseIPRA presentation. One fixed reusable clause, $\mathcal{R} = [\,; y.(y = y)]$, and four inference schemes — Use, Step, Open, and Close — present a theory bi-interpretable and equiconsistent with full first-order intuitionistic primitive recursive arithmetic. Universal reflexivity, ordinary hypotheses, and conditional closures all use the same clause mechanism. Numerical terms are zero-normal scalar iterators; all normalization is included in the checker.

We prove exact translations to and from the repository's five-pattern zero-normal clause calculus, recover its arithmetic and logical interface, and make both interpretation composites explicit. A complete standalone Python checker, bidirectional certificate translators, negative controls, and reproducible source measurements accompany the paper. The selected checker has 2282 normalized bytes, 666 significant tokens, and 770 AST nodes, compared with 2950/829/902 for the pinned transparent five-pattern checker. The comparison includes parsing, syntax and scope validation, hereditary substitution, all rules, and the acceptance interface.

The improvement is a presentation and implementation reduction. Reflexivity remains a counted axiom clause. No new minimum over the repository's compact codecs, no absolute minimality theorem, and no new Lean verification are claimed.

1 Result and comparison boundary

The target is *full first-order* iPRA: intuitionistic first-order logic with equality, a symbol for each fixed primitive-recursive definition, the defining equations, successor separation and injectivity, and induction for quantifier-free formulas. In the full primitive-recursive language, equation induction and quantifier-free induction are interderivable via characteristic functions. The first-order formulation matters: agreement on equations or on provably total functions alone would not establish the requested bi-interpretation. See also Salehi [10] for the first-order equation- induction convention.

The pinned repository already contains scalar iterators, the paired binder D, clause contexts, a single successor Step that recovers equality, and zero-normal syntax [1, 2, 4]. These are inherited ingredients. The new interface shares clause application between three previously different uses, and the implementation removes repeated proof-tree annotations through a goal-directed packet format.

Theorem 1.1 (Main result)**.** *The calculus $\mathcal{C}_4$ specified in Sections 2–3 has the following properties.*

1. *Its public consequences are exactly those of the pinned five-pattern zero-normal clause calculus $\mathcal{C}_5$, under the identity translation on numerical terms and public formulas.*
2. *Its ordinary realization is bi-interpretable with* iPRA*. Both interpretation domains are the entire numerical domain, equality is actual equality, and the two numerical composite isomorphisms have graph $x = y$. Both formula composites are provably equivalent to the original formula.*
3. *$\mathcal{C}_4$ proves its designated contradiction $0 = S0$ if and only if* iPRA *proves falsity. Finite proof translations are effective.*

4. *The delivered standalone checker accepts exactly its closed theorems when run with sufficient resources, with the documented certificate syntax.*

Here "exactly" describes the mathematical checker algorithm on finite certificates. Ordinary Python recursion and memory limits can reject a very large valid certificate. They do not create accepted false conclusions. Neither theory is asserted to prove its own consistency.

How much has actually been simplified? Four inference schemes and one initial axiom clause replace five inference schemes, one of which was reflexivity. Thus counting "four" alone would overstate the result. The substantive sharing is that a single operation handles reflexivity, hypotheses, and conditional clauses. The complete checker comparison, including the initial clause's compiled behavior, is the evidence for a size reduction. A stronger matched control and every selected representation cost are reported in Section 8.

The repository also records a 516-byte compact profile and a 788-byte public-goal validated profile [1, 3]. These have different, heavily fused certificate encodings. The present programs do not improve those numerical records. There is no proved absolute lower bound for all axiomatizations or implementations in this report.

2 Numerical syntax and public formulas

2.1 Zero-normal scalar iteration

There is one numerical sort. Start with raw terms

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only u is bound in h; it is not bound in n or a. Other parameters in the update remain fixed. Define the smart constructor

$$\widehat{I}(n, a; u.h) = \begin{cases} a, & n \text{ is the literal } 0, \\ I(n, a; u.h), & \text{otherwise.} \end{cases}$$

Define N recursively on the syntax: it fixes variables and zero, commutes with S, and rebuilds each iterator with $\widehat{I}$ after normalizing its three children. The actual terms of $\mathcal{C}_4$ are those fixed by N. There is no iterator whose counter is literally zero anywhere in such a term. In a rule, substitution means capture-free substitution followed by N.

This builds in only $I(0, a; u.h) = a$. It does *not* normalize successor iterations, execute an arbitrary primitive-recursive function, or appeal to a numerical evaluator. The counter may be any term, including an iterator. The operation terminates by structural recursion on the finite input tree: the zero contraction returns its already-normalized seed and does not duplicate or reprocess an update.

Lemma 2.1 (Zero-normal substitution)**.** *Normalization is idempotent, does not introduce free variables, and satisfies*

$$\mathsf{N}(t[\sigma]) = \mathsf{N}(\mathsf{N}(t)[\mathsf{N} \circ \sigma]).$$

Consequently hereditary substitution is coherent under composition.

Proof. Induct on the raw term. Variables, zero, and successors are immediate. For an iterator, normalize the counter, seed, and update first. The two sides have equal normalized children by induction. If their common counter is literal zero both return the common normalized seed; otherwise both return the same iterator. Under its update, lift the substitution past its one bound variable. Idempotence is the same structural argument with the identity substitution. No semantic equality test is used. □

2.2 The Eq/D language

Public formulas are

$$A ::= (s = t) \mid D_{x,y}A.$$

The variables x, y bound by D are distinct and are renamed before substitution. The intended ordinary reading is

$$(D_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y\, B^\circ(x,y)\big). \tag{1}$$

Equality is actual numerical equality. The designated contradiction is $\bot_* := (0 = S0)$. There are no primitive symbols for ordinary implication, conjunction, disjunction, universal quantification, or falsity in this native grammar. Their reconstruction is given in Section 6 and the inherited logical appendix.

A useful exact representation. The grammar is a unary spine: every public formula is uniquely $D^k(s = t)$ after fixing a binding convention. Thus it can be represented as a ranked equation (k, s, t). With de Bruijn indices, the endpoints have $2k$ bound numerical slots. This is a bijective encoding, not a prenex normal-form theorem and not a claim that arbitrary ordinary formulas have a single quantifier prefix. Each D has the conditional reading (1). We implemented and tested this representation, but an equally careful recursive Eq/D implementation was smaller. It is therefore reported as a comparison, not used to manufacture a size claim.

3 One fixed clause and four inference schemes

A reusable clause has zero or one antecedent:

$$[A_1, \dots, A_m; y.B(y)], \qquad m \in \{0,1\}, \quad y \notin \bigcup_j \mathrm{FV}(A_j).$$

Its ordinary reading is $\forall y B^\circ(y)$ for $m = 0$ and $A_1^\circ \to \forall y B^\circ(y)$ for $m = 1$. Contexts are finite lists of reusable clauses. Every context has access to the distinguished clause

$$\mathcal{R} = [\ ; y.(y = y)]. \tag{R}$$

An ordinary hypothesis A is the constant unconditional clause $\langle A\rangle := [\ ; y.A]$, with y absent from A. Lists, rather than sets, make context occurrence indices unambiguous in certificates.

The complete rule inventory is the following. All displayed substitutions include the zero normalization of Section 2.

Use. For a retained clause and a well-scoped numerical term t,

$$\frac{[A_1, \dots, A_m; y.B] \in \Gamma \qquad \Gamma \vdash A_1 \quad \cdots \quad \Gamma \vdash A_m}{\Gamma \vdash B(t)}. \tag{Use}$$

The premise list is empty for $m = 0$. In particular Use of $\mathcal{R}$ at t proves $t = t$.

Step.

$$\frac{\Gamma \vdash \widehat{I}(c, a; u.h) = t \qquad \Gamma \vdash I(Sc, a; u.h) = r}{\Gamma \vdash r = h[t/u]}. \tag{Step}$$

The counter, seed, and update agree literally in the two source iterators, up to the built-in zero contraction of the first one. The update is a fixed term template, not a quantified function variable.

Open.

$$\frac{\Gamma \vdash D_{x,y}B(x, y) \qquad \Gamma, [B(b, 0); y.B(b, y)] \vdash A}{\Gamma \vdash A}. \tag{Open}$$

The numerical eigenparameter b is fresh for Γ, A, and all fixed parameters of the displayed matrix. The added clause is discharged.

Close.

$$\frac{\Gamma, \langle B(t, \eta)\rangle \vdash B(t, Sz)}{\Gamma \vdash D_{x,y}B(x, y)}, \qquad \eta = \begin{cases} z, & B \text{ is a literal equation and } t \text{ is literal } 0, \\ 0, & \text{otherwise.} \end{cases} \tag{Close}$$

Here z is fresh for Γ, t, and the fixed parameters of B. The local unconditional clause is discharged. "Literal equation" excludes an equation beneath a D binder. The witness test is a syntactic test, not a test of provable equality with zero.

Nothing else is primitive. There is no other axiom, equality rule, induction rule, logical rule, structural rule, or computation equation. Weakening, renaming, and cut are admissible; they can be proved by translation or the usual induction on derivations, with local eigenparameters renamed. General numerical substitution is admissible via the equivalent unrestricted basis, but should not be justified by blindly copying Close: a substitution can turn a nonzero-looking witness into literal zero and change its selected mode.

4 Recovery of computation, equality, and both introductions

4.1 One successor Step supplies the arithmetic interface

For brevity, write $h(t)$ for normalized substitution. All terms below are well-scoped and zero-normal.

Lemma 4.1 (Computation and the equality pivot)**.** *The calculus derives the successor equation*

$$I(Sc, a; u.h) = h(\widehat{I}(c, a; u.h))$$

and the pivot inference

$$s = t,\ s = r \ \vdash\ r = t.$$

Neither derivation uses induction or a logical rule.

Proof. Successor computation is Step on the two reflexivities supplied by Use of $\mathcal{R}$. For the pivot let $J = I(S0, s; u.u)$. Step on $s = r$ and $J = J$ gives $J = r$. A second Step on $s = t$ and $J = r$ gives $r = t$. □

Symmetry is the pivot on $s = s$ and $s = t$. Given $s = t$ and $t = u$, reverse the first equality and apply the pivot to obtain $s = u$. For transport in one term, assume $s = t$ and $h(s) = v$. Put $K = I(S0, s; u.h)$. Step gives $K = h(s)$ from reflexivities. Transitivity gives $K = v$. Step on $s = t$ and $K = v$ gives $v = h(t)$. Two such term transports give transport in an equation template. Transport in compound formulas follows by induction on the D spine using Open and case introduction.

The zero equation is definitional, with its full normalizer counted in the implementation. The successor equation is derived. Universal reflexivity has not disappeared: its source is exactly clause (R).

4.2 Why the single Close rule is sufficient

The familiar case introduction would allow $\Gamma, B(t, 0) \vdash B(t, Sz)$ for every t; equation progress would allow $\Gamma, E(t, z) \vdash E(t, Sz)$. Close selects one mode deterministically. The apparent missing atomic case at literal witness zero is recoverable.

Let

$$q = I(S0, 0; u.u).$$

It is not syntactically zero, and Lemma 4.1 gives $q = 0$. Suppose $\Gamma, E(0, 0) \vdash E(0, Sz)$. First use Close on the matrix $E(0, y)$ with an unused witness coordinate and witness q. This uses case mode, and gives $D_{v,y}E(0, y)$. Open it to obtain the clause $[E(0, 0); y.E(0, y)]$. Now introduce $D_{x,y}E(x, y)$ with witness q: its assumption $E(q, 0)$ transports to $E(0, 0)$; Use supplies $E(0, Sz)$; transport back gives $E(q, Sz)$. Close and then Open discharge the added data. The desired atomic case conclusion follows.

Equation progress for a fixed matrix $E(t, y)$ is introduced first with an unused witness coordinate and literal witness zero. Its Open clause gives the conditional closure of $E(t, y)$. This supplies every witnessed progress instance through the now unrestricted case rule. In particular, from

$$\Gamma \vdash E(0), \qquad \Gamma, E(z) \vdash E(Sz)$$

one derives $\Gamma \vdash E(t)$ for any t by Close, Open, and Use. Numerical cases for arbitrary formulas are obtained in the same way from their zero proof and fresh successor proof using case mode.

These constructions explain why Close is sufficient and why extending progress to arbitrary compound matrices would be a different theory. The latter would give induction for arbitrary formulas by exactly the same Open–Use argument.

5 Exact proof equivalence with the pinned calculus

The matched source $\mathcal{C}_5$ uses the same zero-normal terms, public Eq/D formulas, Step, Open, and witness-selected Close. It has explicit reflexivity and conditional clause application. A local ordinary hypothesis A is stored as $[0 = 0; y.A]$ and needs a proof of its neutral antecedent. This is the complete checker in `Research/uniform_step/code/checker.py` at the pinned commit.

Theorem 5.1 (Clause factorization)**.** *There are effective translations of finite proofs in both directions between $\mathcal{C}_5$ and $\mathcal{C}_4$, preserving each public conclusion and retained side assumptions under the clause translations below. In particular, their closed theorem sets are identical.*

Proof. For the forward translation, insert $\mathcal{R}$ as the first new context occurrence. Replace source reflexivity at t by Use of that occurrence at t with no premises. Mark each source context occurrence according to whether Open or Close introduced it. Open occurrences remain conditional. A Close occurrence $[0 = 0; y.A]$ becomes $\langle A \rangle$; a Use of that occurrence discards the source proof of $0 = 0$. Translate other premises recursively. Step, Open, and Close retain the same public formulas and witnesses.

For the reverse translation, a Use of the distinguished $\mathcal{R}$ becomes source reflexivity. Every other unconditional clause $[\ ; y.B]$ is represented by $[0 = 0; y.B]$, and its application gets the source proof $0 = 0$ by reflexivity. Conditional clauses and their applications are unchanged. Again translate the other rules recursively.

The induction tracks context *occurrences*, not only their formula contents. Therefore equal-looking clauses do not confuse the conversion, and discharge and eigenparameter conditions are preserved. No equality proof, computation, or logical rule is imported as an oracle. The proof does not need substitution stability of the witness-selected rule. □

Representation changes. Removing the numerical constructor tags is a bijection: St becomes $[t]$ and $I(n, a; u.h)$ becomes $[n, a, h]$. Public equality becomes the endpoint pair $[s, t]$; the paired formula remains $[\mathtt{d}, B]$. The types make these shapes disjoint: a numerical term is an integer or a list of length one or three, and a formula is a list of length two. The tag $\mathtt{d}$ is never a numerical term. Hence one opening routine can traverse all these objects with the correct binder increments. Structural induction proves that the encoding commutes with normalized opening, so it transports the entire proof system without semantic change.

6 Full intuitionistic logic and the bi-interpretation

6.1 The ordinary realization

Let T_I be intuitionistic first-order logic with equality over the staged scalar iterator language. For each fixed earlier-stage update $h(u, \vec{p})$, introduce a symbol $i_h(n, a, \vec{p})$ with

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Add numerical coverage $\forall x(x = 0 \vee \exists y\, x = Sy)$, the separation bridge $(0 = S0) \to \bot$, and contextual induction for literal equations. This uses fixed finite function descriptions, with no uniform evaluator over function codes.

Interpret $\mathcal{C}_4$ by actual equality, (1), and the clause readings in Section 3. Use and Open are ordinary logical inferences. Step follows from the defining equation and equality. Case Close is sound by coverage; its progress case is sound by equation induction under a temporarily assumed base. The initial clause is equality reflexivity. This proves soundness for the *whole* ordinary domain.

The explicit separation bridge is essential in the ordinary theory: positive iterator equations alone admit a one-element structure. Native falsity is $0 = S0$; ordinary falsity is $\bot$. Equiconsistency must

preserve this distinction.

6.2 Logical macros and their composites

Set $C(v, a, b) = I(v, a; u.b)$, with u unused, so $C(0, a, b) = a$ and $C(Sv, a, b) = b$. The gate $G_v A$ replaces a terminal equation $s = t$ by $C(v, 0, s) = C(v, 0, t)$ and commutes with D. Thus $G_0 A$ is derivable and $G_{Sv} A$ is interderivable with A. Padding $A \dashv\vdash D_{u,y} G_y A$ aligns formula depths.

Using this padding, define a selector $M_v(A, B)$ with $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$: at equations use C on both endpoints, and at equal D depths recurse under their aligned binders. When one operand is an equation and the other has an outer D, pad the equation with G_y and recurse under that D. The maximum input depth decreases at each call. Let $p(z) = I(z, 0; u.C(u, 1, 0))$; fixed computations give $p(1) = 1$, $p(2) = 0$. Then

$$\begin{aligned} A \Rightarrow_* B &= D_{u,z} M_z(A, B), \\ A \wedge_* B &= D_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x A(x) &= D_{x,y} G_y A(x), \\ A \vee_* B &= \exists_* x M_x(A, B), \\ \forall_* x A(x) &= A(0) \wedge_* D_{u,z} G_z A(z), \\ \bot_* &= (0 = 1). \end{aligned}$$

All added bound variables are fresh, and u is unused. The inherited logical appendix supplies the introduction and elimination derivations. For example, implication's zero instance supplies its assumption and its successor instances supply the consequent. Existential introduction uses the chosen witness and the gate; Open, the true zero gate, and Use at 1 give existential elimination. Numerical cases supply disjunction elimination and the missing zero case of universal elimination.

For the crucial native formula composite one proves

$$D_{x,y} B \dashv\vdash \exists_* x \big(B(x, 0) \Rightarrow_* \forall_* y B(x, y) \big).$$

Forward, Open supplies its conditional clause; introduce the three logical macros around that clause. Backward, eliminate the existential at a fresh witness and apply case Close using implication and universal elimination. Induction on finite formulas proves both logical composites. This is more than numerical conservativity.

6.3 Arithmetic translations and identity composites

In iPRA, interpret a scalar iterator by primitive recursion whose update ignores the recursion counter. Conversely, the inherited scalar development establishes, in this order: a predecessor by equation induction and cases; truncated subtraction and a characteristic of actual equality; quantifier- free induction; and pairing with projections. The detailed proofs are reproduced in the arithmetic appendices [7, 8, 9]. This order avoids assuming pairing or equality reflection in the predecessor bootstrap.

For a primitive recursion

$$f(0, \vec{p}) = g(\vec{p}), \qquad f(Sn, \vec{p}) = h(n, f(n, \vec{p}), \vec{p}),$$

let π, L, H be the derived pairing and projections. Its scalar translation is

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))),$$
$$f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the projection equations then prove both defining equations for f^*. Initial functions and composition are direct. This is external recursion on each finite definition.

Two scalar iterators with equal seeds and provably equal updates are equal at every internal counter by equation induction. The analogous uniqueness proof works for ordinary primitive recursion. External induction on a finite function description therefore proves that its double translation equals the original operation internally. The interpretations use the whole domain and actual equality; $x = y$ is consequently a provably total bijective graph preserving all operations in both composites. Combine this with the preceding formula composites and the falsity bridge to prove Theorem 1.1(2)–(3).

The repository formalizes the inherited bridge in `BiInterpretation.lean` and `NativeBiInterpretation.lean`; its zero-normal Step interface is `NativeZeroStep.lean` [5, 6, 4]. We inspected the pinned sources. We did not rebuild that Lean project in this run. The new clause factorization and packet checker correctness here are written proofs with executable cross-checks, not additional Lean theorems.

7 The complete checker and its certificates

7.1 Binding and validation

The selected checker uses exact integers and lists. Zero is integer 0; positive integers are de Bruijn indices; negative integers are fresh free levels. At numerical scope q and bound depth d, an integer is valid exactly when $-q \le a \le d$. Thus Python booleans and floats are rejected even though some of them compare equal to integers. Successor is $[t]$; an iterator is $[n, a, h]$, with $n \neq 0$ syntactically and the body checked at depth $d+1$. A formula is $[s, t]$ or $[\mathtt{d}, B]$; its D matrix is checked at depth $d+2$.

In a paired matrix, index 1 denotes y and index 2 denotes x, before accounting for any further inner binders. At each reachable packet checker state, the numerical scope equals the length of the explicit context: the fixed $\mathcal{R}$ is ambient, and each Open or Close adds exactly one clause and one new numerical level. This invariant removes a separate scope argument without identifying arbitrary contexts with variable scopes.

Opening at cutoff d replaces index $d+1$ by the supplied term, decreases larger indices by one, and leaves smaller indices and negative levels unchanged. It increments the cutoff by one under an iterator body and by two under D. Rebuilding an iterator with counter zero returns its seed. Inserted terms have no free positive indices, so they require no additional lifting; their own internal bound indices remain protected. This is a locally nameless representation with stable negative free levels.

Every intermediate goal is validated before a rule is processed. For Open, its conclusion is checked before extending the scope; it cannot mention the new eigenparameter. Close validates its witness in the old scope. Context clauses are generated only by those rules, so their validity is maintained internally rather than trusted from input.

7.2 Goal-directed packets

The selected complete certificate is a JSON array of two-field packets followed by a closed goal. Packets are read from right to left. A packet $[b, t]$ selects its inference from the shape of b:

First field b	Meaning of the packet
Nonnegative integer	Use context clause b at numerical target t.
-1	Close the current paired goal with witness t.
A paired formula	Prove b, Open it, then prove the retained goal; the unused second field must be the exact integer 0.
An equation	Prove b and the equation t, then check that they are the two premises of Step for the current goal.

A reflexive equation closes immediately by Use of $\mathcal{R}$, without consuming a packet. The fixed clause is therefore compiled into a reflexivity test; its behavior remains counted in the checker. Context indices enumerate the local clauses, since $\mathcal{R}$ needs no index in this protocol. All consumed instructions are checked, and any unconsumed prefix causes rejection. The companion tree checker implements the explicit four-scheme formulation with a stored initial clause and a finite list of premises for Use.

Neutral-antecedent lowering. An unconditional local clause can be stored internally as a clause with antecedent $0 = 0$: the check of that antecedent closes automatically without consuming input. This is a compilation of empty premise lists, not an additional axiom and not an omitted proof obligation. It permits the same premise-checking path for every local clause. The comparison includes this implementation choice.

Theorem 7.1 (Packet checker correctness)**.** *Successful checking of a goal under a well-scoped context yields a finite $\mathcal{C}_4$ derivation of that goal. Every finite $\mathcal{C}_4$ derivation of a closed theorem, with no ordinary assumptions apart from the ambient $\mathcal{R}$, has an accepted packet certificate with the same conclusion.*

Proof. For soundness, induct on the finite successful call tree. Automatic closure is Use of $\mathcal{R}$. Otherwise one packet is removed; its disjoint shape chooses one of the four rules. Recursive calls establish its premises, and the explicit comparisons establish its conclusion and all side conditions. The pre-extension validation of an Open conclusion gives the eigenvariable condition. Clause instances use the proved hereditary-opening operation. Neutral-antecedent lowering uses the fixed reflexivity proof of $0 = 0$. The other validation invariants follow by induction on the context.

For completeness, induct on a derivation. Replace any proof of a reflexive goal by Use of $\mathcal{R}$. At a nonreflexive goal, emit the packet for its last rule and serialize its premise proofs in the order in which the checker will request them. Reverse this instruction sequence in the external array and append the goal. The suffix-consumption invariant says that one call consumes exactly its own serialized proof segment and leaves the preceding segments intact. Prove it by the same induction. Empty-premise Use consumes no premise segment; its lowered neutral proof also consumes none. Every rule is represented, and at the root the remaining array is empty. □

This proves correctness of the algorithm in the paper's metatheory. Finite tests support the implementation correspondence but cannot replace this universal proof.

8 Measurements, validation, and limits

All source measurements use CPython 3.12.14, remove documentation strings, apply `ast.unparse`, and append one final LF. Significant tokens exclude comments, layout, encoding, and end markers. AST counts include the complete parsed acceptance module. Gzip uses level nine and timestamp zero; it is informational. The exact measured sources and gzip files are included, together with their hashes and the measurement script.

Complete checker	Bytes	Tokens	AST	Gzip 9
Pinned five-pattern tagged checker	2950	829	902	1005
Five-rule arity and scope control	2730	753	828	939
Four-rule explicit tree checker	2677	751	823	947
Four-rule ranked tree checker	2692	781	853	964
Four-rule empty-antecedent packets	2308	665	770	778
Selected four-pattern packet checker	2282	666	770	774

The five-pattern arity/scope control is deliberately stronger than the unaltered tagged source: it receives the same routine codec and scope improvements. In that comparison, clause factorization alone saves only 53 bytes, 2 significant tokens, and 5 AST nodes; its gzip size grows by 8 bytes. The larger reduction comes from the full certificate protocol and shared validation, not from pretending that a relocated axiom vanished.

The exact ranking representation also received a fair control. Ranking with arity-coded terms has 2692/781/853 in the scope-derived implementation, whereas recursive formulas with the same term encoding have 2677/751/823. Thus rank flattening is a useful structural observation, but does not win this comparison. The report selects the smaller verified implementation.

The selected packet protocol passed 620 standalone process checks, using normal Python and `python -O`. These include 86 accepted certificates, 86 exact packet round trips, and 52 rejection cases. A separate 2101 generated semantic checks compared opening with an independent finite numerical interpreter. There were zero failures. Additional tree-format and ranked-format suites check the intermediate translations; their separate results are included rather than added to the selected count.

Checks include hypotheses that are genuinely used, dependent paired elimination, equation progress with a used induction hypothesis, witnesses with nested iterator binders, malformed integers and lists, scope escapes, forged Step premises, changed iterator bodies, missing clause premises, and leftover packet input. The source control is frozen at the pinned commit; its SHA-256 is

`17d116f7a6cbccaca169f76124aed5a9b20d45818747f96fba1c6f36668ad296.`

What remains open. This work supplies a smaller complete checker in a transparent comparison family and a four-scheme presentation with an explicit initial clause. It does not establish a globally smallest axiomatization, does not improve the repository's compact 516-byte frontier, and does not assert novel arithmetic strength. A future improvement should either delete genuinely independent trusted structure or improve a matching complete implementation; counting rule names or omitting typing and substitution costs is insufficient.

9 A further logical quotient: rank from variable support

There is a genuine logical quotient behind a tempting further simplification. If the outer two variables of $D_{x,y}B$ are both absent from B, then

$$D_{x,y}B \;\leftrightarrow\; \exists x(B \to B) \;\leftrightarrow\; \top.$$

For a ranked formula (k, s, t), let r be the largest free positive de Bruijn index in the two endpoints, counted relative to iterator binders. Take $r = 0$ when the support is empty. If $\lceil r/2 \rceil < k$, its outer pair is vacuous and the formula can be replaced by $0 = 0$. Otherwise $k = \lceil r/2 \rceil$ and the public rank can be reconstructed from the endpoints.

This does not remove ranks everywhere. A clause body has an additional external universal slot at index $2k + 1$. The same endpoint support can then mean a rank-zero body with a clause variable or a public rank-one formula. Internal clause ranks must still be retained unless the variable convention is changed. The implemented support experiment grew to 3069 normalized bytes, with 854 tokens and 957 AST nodes. It is not selected. Its source is supplied as an exploratory size comparison and is outside the verified checker suite. The optional proof notes give the quotient argument and the elimination case; the selected checker uses the exact Eq/D syntax and needs none of that extra machinery.

10 Reproduction and artifact guide

Run `python3 reproduce.py` to execute the selected translations and checks and regenerate measurements. Run `python3 reproduce.py --pdf` to compile the paper as well. The selected checker runs directly on a certificate with only the Python standard library. The accompanying README gives its exact path and sample command, and records the compiler and verification environment.

The delivered `report.tex` is self-contained: its checker listing and inherited appendices are embedded. The source fragments and build script are also supplied for editing. The inherited mathematical material below is attributed to the pinned repository and is included to make the arithmetic bridge inspectable offline. It is not presented as a new result of this iteration.

A Complete selected checker

The following is the complete measured source. Imports, parsing, rejection, scope checks, zero normalization, rule processing, and final acceptance are all included.

```
import json

def require(condition):
    if not condition:
        raise ValueError('Invalid certificate')

def term(a, scope, depth=0):
    if type(a) is int:
        return -scope <= a <= depth
    return type(a) is list and len(a) in (1, 3) and (len(a) == 1 or a[0] != 0) and all((term(t, scope, depth
     + (len(a) == 3 and i == 2)) for i, t in enumerate(a)))

def formula(a, scope, depth=0):
    return type(a) is list and len(a) == 2 and (formula(a[1], scope, depth + 2) if a[0] == 'd' else all((term
     (t, scope, depth) for t in a)))

def opening(a, value, depth=0):
    if type(a) is int:
        return value if a == depth + 1 else a - (a > depth + 1)
    if a[0] == 'd':
        return ['d', opening(a[1], value, depth + 2)]
    b = [opening(t, value, depth + (len(a) == 3 and i == 2)) for i, t in enumerate(a)]
    return b[1] if len(a) == 3 and b[0] == 0 else b

def check(goal, context=()):
    scope = len(context)
    require(formula(goal, scope))
    if goal[0] == goal[1]:
        return
    major, data = packets.pop()
    fresh = -scope - 1
    if type(major) is int:
        if major >= 0:
            require(major < len(context) and term(data, scope))
            antecedent, body = context[major]
            require(goal == opening(body, data))
            check(antecedent, context)
        else:
            require(major == -1 and goal[0] == 'd' and term(data, scope))
            body = opening(goal[1], data, 1)
            eta = fresh if goal[1][0] != 'd' and data == 0 else 0
            check(opening(body, [fresh]), context + (([0, 0], opening(body, eta)),))
    else:
        check(major, context)
        s, t = major
        if s == 'd':
            require(data == 0 and type(data) is int)
            body = opening(t, fresh, 1)
            check(goal, context + ((opening(body, 0), body),))
        else:
            check(data, context)
            match data:
                case [[[n], a, h], r]:
                    require(s == (a if n == 0 else [n, a, h]) and goal == [r, opening(h, t)])
                    return
            raise ValueError('Invalid step')
if __name__ == '__main__':
    packets = json.load(open(0))
    require(type(packets) is list and bool(packets))
    check(packets.pop())
    require(not packets)
    print('ACCEPT')
```

Inherited foundations: provenance

The following five appendices reproduce the pinned `Research/certified_zero/reference/` foundations under the upstream MIT-0 license. Their references to an "old" or "new" choice rule concern that earlier basis. The identity-language equivalence in the present Section 5 and the recovery in Section 4 connect that basis to the four-scheme calculus. The appended proofs supply the logical interface, ordinary realization, predecessor, equality reflection, and primitive-recursion compiler.

B Complementary introductions with one local assumption

The old paired introduction retains both the zero instance and an induction instance when its matrix is an equation. Both together are unnecessary. A certificate may choose either one, provided the induction choice is restricted to a literal equation. This changes neither the formula language nor the meaning of a formula. The omitted combination is recovered by a short nested introduction–elimination construction.

B.1 Complete native syntax and rule inventory

The scalar numerical language is

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only h binds the state variable u; n and a do not. The update may contain fixed numerical parameters. The computation axioms in every retained context are

$$I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{I0, IS}$$

The argument also applies unchanged to the full fixed-description PR language with its projection, composition, and recursion equations. No numerical variable ranges over function descriptions, and no uniform evaluator is postulated.

Native formulas and reusable context clauses are

$$A ::= (s = t) \mid \mathsf{D}_{x,y}A, \qquad \langle A; y.B(y)\rangle, \quad y \notin \mathrm{FV}(A).$$

Bound variables are renamed before capture-free substitution. The two variables x, y in a paired formula are distinct. An ordinary assumption F is the constant clause $\langle H; v.F\rangle$, where v is absent from F and H is the fixed computation theorem $I(0, 0; u.0) = 0$. Thus ordinary hypothesis use is a clause Use preceded by this computation proof.

There are computation axioms and exactly the following inference patterns:

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)} \ \mathsf{Use} \quad (\langle A; y.B(y)\rangle \in \Gamma),$$

$$\frac{\Gamma \vdash \mathsf{D}_{x,y}B(x, y) \qquad \Gamma, \langle B(a, 0); y.B(a, y)\rangle \vdash F}{\Gamma \vdash F} \ \mathsf{DE}, \qquad \frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)} \ \mathsf{Tr},$$

$$\frac{\Gamma, B(t, \eta_\varepsilon(z)) \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)} \quad \mathsf{DI}_\varepsilon, \qquad \eta_0(z) = 0, \quad \eta_1(z) = z. \tag{Choice DI}$$

Here ε is a certificate choice in $\{0, 1\}$, and $\varepsilon = 1$ is permitted only when B has a literal equation root. An equation beneath a binder is not a literal equation. Call $\varepsilon = 0$ the *case choice* and $\varepsilon = 1$ the *progress choice*. Both retain exactly one ordinary formula assumption. Compound matrices have only the case choice.

In DI the eigenparameter z is fresh for Γ, the witness t, and every fixed parameter of B. In DE the eigenparameter a is fresh for Γ, the conclusion F, and every fixed parameter of B. In transport, $F(v)$ is any fixed native formula template. Displayed local assumptions and clauses are discharged. There are no primitive reflexivity, coverage, separation, ordinary logical, or additional induction rules.

We call this calculus $\mathcal{C}$. The original comparison calculus $\mathcal{N}$ has identical syntax, computation, Use, DE, and transport, but its introduction is

$$\frac{\Gamma, B(t, 0), [B(t, z)]_{\mathrm{at}} \vdash B(t, Sz)}{\Gamma \vdash \mathsf{D}_{x,y} B(x, y)}. \tag{Old DI}$$

The bracket supplies $B(t, z)$ exactly at a literal equation.

B.2 Exact equivalence and the unused-witness construction

Lemma B.1 (Structural admissibility)**.** *Weakening, exchange, contraction, renaming, ordinary-assumption cut, and capture-free numerical substitution are admissible in $\mathcal{C}$.*

Proof. Induct on derivations. Rename local eigenparameters before inserting proofs or terms. Numerical substitution preserves a formula's root constructor, so an allowed progress choice remains allowed. The selected choice does not change under substitution or cut. For ordinary-assumption cut, replace each use of its constant clause by the supplied proof. Computation schemas are closed under capture-free numerical substitution. These facts verify every case. □

Theorem B.2 (Identity-language equivalence)**.** *For every retained clause context Γ and native formula F,*

$$\Gamma \vdash_{\mathcal{C}} F \quad \Longleftrightarrow \quad \Gamma \vdash_{\mathcal{N}} F.$$

The transformations are effective on finite derivations and leave all numerical and formula syntax unchanged.

Proof. Each new introduction is an instance of the old one after weakening by the omitted local assumption. For the case choice at an equation, add the induction instance. For the progress choice, add the zero instance. Compound case introduction is already the old rule. Thus every new derivation translates to an old one.

For the reverse direction, transform an old derivation recursively. Only its atomic DI case needs a construction. Suppose its matrix is the equation $E(x, y)$ and its transformed premise is

$$\pi : \quad \Gamma, E(t, 0), E(t, z) \vdash E(t, Sz).$$

The outer new inference will be case DI with witness t. It is therefore enough to prove $E(t, Sz)$ under $\Gamma, E(t, 0)$.

Rename the eigenvariable in π to a fresh v. Choose a new variable u absent from $E(t,y)$ and all retained data. Under $\Gamma, E(t,0)$, progress DI with witness zero introduces

$$\mathsf{D}_{u,y}E(t,y)$$

from exactly the renamed premise $\pi[v/z]$. The bound coordinate u is deliberately unused.

Apply DE to this paired formula. Because u is unused, the opened clause is exactly

$$\langle E(t,0); y.E(t,y)\rangle;$$

it contains no new existential witness. The retained ordinary assumption $E(t,0)$ supplies its antecedent. Use at Sz gives $E(t,Sz)$, and DE closes. Finally the outer case DI introduces the original $\mathsf{D}_{x,y}E(x,y)$ with witness t. The original premise proof π has been inserted once.

An old compound introduction already is the new case choice. All other rule cases are unchanged. This completes the induction on the original derivation. □

The unused inner coordinate is essential. Introducing $\mathsf{D}_{x,y}E(x,y)$ internally and then eliminating it would give an arbitrary fresh witness, not necessarily the original t. The construction instead keeps t as a fixed parameter of $E(t,y)$ and binds only the unused coordinate u.

Proposition B.3 (Linear bound on inference nodes)**.** *Count each computation axiom, Use, DI, DE, and transport as one inference node, expanding ordinary-assumption use as a computation of H followed by Use. If an old derivation has N nodes and r atomic DI nodes, the displayed transformation has exactly*

$$N + 5r \leq 6N$$

nodes.

Proof. Each atomic old DI node is replaced by six nodes: outer case DI, inner progress DI, DE, Use of the opened clause, Use of the retained base assumption, and the computation of H needed for that last ordinary-assumption use. Its old premise subtree occurs once. All other inference nodes are copied once. □

This counts inference-tree nodes. It is not a bound on serialized JSON bytes, formula sizes, checker runtime, or the number of consumed packet cells when hypothesis closure is implicit. In the formula-context Q/D presentation, the corresponding replacement has outer case DI, inner progress DI, DE, QE, and two Hyp nodes, giving the same six-node count.

B.3 Induction and numerical cases separate cleanly

Lemma B.4 (Contextual equation induction)**.** *The progress choice, DE, and Use derive*

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)} \tag{EI}$$

for every equation E, with the usual freshness condition.

Proof. Choose an unused witness coordinate u. Progress DI introduces $\mathsf{D}_{u,y}E(y)$ from the given step. DE opens the clause $\langle E(0); y.E(y)\rangle$. Supply its antecedent using the given base proof and use the clause at t. Close DE. □

Lemma B.5 (Cases, equality, and binder congruence)**.** *The case choice derives, for arbitrary native formulas,*

$$\frac{\Gamma \vdash F(0) \qquad \Gamma \vdash F(Sz)}{\Gamma \vdash F(t)} \tag{NC}$$

with the usual freshness condition. Equality reflexivity, symmetry, transitivity, numerical congruence, and uniform matrix interderivability under D *are derivable without using the progress choice.*

Proof. For cases, use case DI with an unused witness coordinate and matrix $F(y)$, ignoring its retained base in the successor premise. DE opens $\langle F(0); y.F(y)\rangle$; the given zero proof and Use at t finish.

For reflexivity put $r = I(0, t; u.0)$. Use the computation equation $r = t$ as both transport premises with motive $v = t$ to obtain $t = t$. The usual explicit equality motives give symmetry, transitivity, and numerical congruence.

For binder congruence, open the old paired formula at a fresh witness a and introduce the new one by case DI at a. Convert its retained base to the old base, use the opened old clause at the successor, and convert back. Ordinary cut closes the construction, and the reverse conversion gives the converse. No induction is used. □

The primitive progress choice is still restricted to equations. Allowing it at arbitrary compound matrices would, by the argument for (EI), supply induction for arbitrary formulas and change the intended theory.

B.4 The entire ordinary logical interface

Here is a predecessor-free reconstruction of the ordinary logical operations. It makes explicit that the case choice supplies the logical interface independently of progress induction.

Let $C(v, a, b) := I(v, a; u.b)$ with u unused. Computation gives $C(0, a, b) = a$ and $C(Sv, a, b) = b$. Define the gate on fixed finite formula syntax by

$$G_v(s = t) := (C(v, 0, s) = C(v, 0, t)), \qquad G_v(\mathsf{D}_{x,y}B) := \mathsf{D}_{x,y}G_vB,$$

renaming bound variables away from the control. Equality, binder congruence, and external structural induction give

$$\vdash G_0A, \qquad G_{Sv}A \dashv\vdash A, \qquad A \dashv\vdash \mathsf{D}_{u,y}G_yA \quad (u, y \notin \mathrm{FV}(A)). \tag{Gate and padding}$$

For padding, case DI proves the forward direction. DE, the zero-gate theorem, and Use at 1 prove the reverse direction.

Define a selector between two arbitrary formulas by

$$M_v(s = t, u = w) := (C(v, s, u) = C(v, t, w))$$

and, with aligned fresh paired variables, by

$$\begin{aligned} M_v(\mathsf{D}_{x,y}A, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(A, B),\\ M_v(E, \mathsf{D}_{x,y}B) &:= \mathsf{D}_{x,y}M_v(G_yE, B),\\ M_v(\mathsf{D}_{x,y}A, E) &:= \mathsf{D}_{x,y}M_v(A, G_yE), \end{aligned}$$

where E is an equation. Every call decreases maximum input binder depth. Padding and congruence prove $M_0(A, B) \dashv\vdash A$ and $M_{Sv}(A, B) \dashv\vdash B$.

Put $\delta(v) = C(v, 1, 0)$ and $p(z) = I(z, 0; u.\delta(u))$. Only the fixed computations $p(1) = 1$ and $p(2) = 0$ are needed. The full logical macro inventory is

$$\begin{aligned} A \Rightarrow_* B &:= \mathsf{D}_{u,z} M_z(A, B), \\ A \wedge_* B &:= \mathsf{D}_{u,z} G_z M_{p(z)}(A, B), \\ \exists_* x\, A(x) &:= \mathsf{D}_{x,y} G_y A(x), \\ A \vee_* B &:= \exists_* x\, M_x(A, B), \\ \forall_* x\, A(x) &:= A(0) \wedge_* \mathsf{D}_{u,z} G_z A(z), \\ \bot_* &:= (0 = 1). \end{aligned} \tag{Logical macros}$$

Newly bound variables are fresh; u is unused.

Implication introduction obtains its assumption from the zero selector and supplies the consequent at successors, then uses case DI. Elimination opens its clause and uses it at 1. Existential introduction uses its witness and the successor gate; elimination opens at a fresh witness, supplies the zero-gate theorem, and uses the clause at 1.

Disjunction introduction uses witnesses 0 and 1. For elimination, discharge the two branch proofs as $A \Rightarrow_* F$ and $B \Rightarrow_* F$. NC applied to $M_z(A, B) \Rightarrow_* F$ gives this implication at every control: the two endpoint cases follow from the branch implications. Existential elimination provides $M_a(A, B)$ and the implication gives F.

Given A and B, selector endpoints and NC give $M_v(A, B)$ at an arbitrary control. Substitute $v = p(Sz)$ and use the successor gate for conjunction introduction. For projections, open its zero-gated clause and specialize at 1 and 2; the two fixed computations give B and A, respectively.

These rules prove existential numerical coverage:

$$(x = 0) \vee_* \exists_* v\, (x = Sv). \tag{Coverage}$$

Use NC. Zero takes the left injection of reflexivity, and a successor takes the right injection with the evident witness. Coverage gives equality cases even when the retained context contains x; its elimination adds $x = 0$ or $x = Sv$ rather than silently changing that retained context.

Universal introduction follows from a fresh-parameter proof by substituting zero and successors and forming the two conjuncts. For elimination, take its $A(0)$ conjunct and open the paired conjunct. Its base is a zero-gate theorem. Coverage of the target chooses the zero instance from $A(0)$ or a successor instance from the opened clause and gate removal.

Finally, under $0 = 1$, computation and transport give $s = C(0, s, t) = C(1, s, t) = t$ for arbitrary numerical terms. Every equation follows, and external induction on formula syntax with case DI gives every native formula. This supplies falsity elimination. No excluded middle for quantified formulas has entered the construction.

B.5 Whole-domain realizations and both formula composites

For a standard first-order reading, let T_{eq} have intuitionistic equality logic, the same numerical signature and computation equations, numerical coverage, separation $(0 = 1) \to \bot$, and contextual

equation induction. Its interpretation of the native syntax is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x\big(B^\circ(x,0) \to \forall y B^\circ(x,y)\big),$$

$$\langle A; y.B\rangle^\circ = A^\circ \to \forall y B^\circ.$$

The domain is the entire numerical domain and interpreted equality is actual equality.

Case DI is sound in this realization by numerical coverage. Progress DI is sound by contextual equation induction under the conditional base. Use, DE, transport, and computation have their displayed ordinary meanings. Conversely, the native macros derive every ordinary logical rule, (EI) derives equation induction, and (Coverage) derives coverage. Separation translates to the identity implication $(0 = 1) \Rightarrow_* (0 = 1)$.

For the essential paired formula composite,

$$\mathsf{D}_{x,y}B \dashv\vdash \exists_* x\big(B(x,0) \Rightarrow_* \forall_* y\, B(x,y)\big). \qquad \text{(Paired composite)}$$

Forwards, DE opens its clause at a fresh a; under $B(a,0)$ that clause supplies $B(a,y)$ at a fresh arbitrary y. Introduce the universal, implication, and existential macros and close DE. Backwards, eliminate the existential at a fresh a. Introduce the paired formula with witness a by case DI: the retained base, supplied implication, and universal elimination at Sz give its successor premise.

The ordinary interpretations of the six logical macros have their ordinary round trips. For conjunction, its true base and the instances $1, 2$ recover both operands; the converse uses selector cases. The universal has all successor instances in its paired conjunct and zero in its first conjunct; coverage gives exactly the ordinary universal. Implication, disjunction, and existential quantification use their selector or gate endpoints. Falsity uses the explicit ordinary separation bridge. External induction on finite formulas, together with (Paired composite), gives both formula composites. Clauses correspond in both directions to $A \Rightarrow_* \forall_* yB$ by the derived logical rules.

The numerical composites are literal identities with internally provable graph $x = y$. No standard-numeral restriction, retract, quotient, or equality reflection assumption is being inserted at this stage. The separately supplied arithmetic bridge identifies T_{eq} with full iPRA; the present change preserves that bridge unchanged because Theorem B.2 preserves every native sequent.

B.6 What the theorem does and does not claim

The new inference interface retains one ordinary assumption at every paired introduction. It preserves the entire consequence relation, not just equations or closed theorems. Consequently the established full-iPRA interpretations and equiconsistency transfer unchanged.

This result is different from retaining only progress introduction and strengthening closure elimination. Here the original elimination rule is retained, and the case and progress introductions coexist. It is also different from allowing arbitrary compound progress, which would supply stronger induction.

A complete-checker comparison must still count the certificate choice, its equation-root restriction, substitution, context handling, computation, and the public input/output interface. The mathematical theorem alone is not a byte-count or minimality theorem. The inference-node bound is distinct from source-size measurements and from serialized certificate size. The proof is a written universal proof; finite regression tests do not constitute a Lean/Rocq formalization of it.

C Arithmetic strength and both ordinary interpretation composites

We spell out the arithmetic bridge to avoid identifying standard numerical agreement with bi-interpretability. First elaborate every fixed iterator into an ordinary function symbol. At stage zero the symbols are $0, S$. Given a fixed earlier-stage term $h(u, \vec{p})$, introduce $i_h(n, a, \vec{p})$ with the two equations

$$i_h(0, a, \vec{p}) = a, \qquad i_h(Sn, a, \vec{p}) = h(i_h(n, a, \vec{p}), \vec{p}).$$

Only finite descriptions occur; the domain contains no code for a universal evaluator. Formal parameters are distinct when a description is declared, while actual arguments can coincide.

Let T_I be ordinary intuitionistic first-order logic with actual equality and genuine falsity, these staged equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot.$$

The second sentence is the falsity bridge. It must be stated: positive numerical equations by themselves admit a singleton structure. Native explosion from $0 = S0$ and a translation of ordinary falsity to that equation do not authorize silently dropping the bridge from T_I.

The native-to-ordinary translation is

$$(s = t)^\circ = (s = t), \qquad (\mathsf{D}_{x,y}B)^\circ = \exists x(B^\circ(x, 0) \to \forall y B^\circ(x, y)).$$

A clause $\langle A; y.B\rangle$ becomes $A^\circ \to \forall y B^\circ$. For the auxiliary closure used by the compact checkers, $(\mathsf{Q}_y B)^\circ = B^\circ(0) \to \forall y B^\circ$. The interpretation domain is all elements, defined by $x = x$; equality is actual equality.

Case DI is sound because numerical coverage combines its retained base with its successor proof. Progress DI is sound by literal-equation induction under a temporarily assumed base. Use, DE and transport are ordinary logical inferences under the translations. Conversely the derived logical operations, equation induction, cases and explosion in Section B recover the rules and axioms of T_I. An auxiliary closure is interderivable with $\mathsf{D}_{v,y}B(y)$ for unused v: in one direction use case DI with the closure and its base; in the other eliminate the paired formula and return its supplied closure. Thus formulas and context clauses have the stated meanings in both presentations.

C.1 The arithmetic dependency order

The scalar update has no recursion-counter argument. We do not conceal one in an encoding. The inherited internal proofs, reproduced in the appendices, establish the required arithmetic in this order:

1. Using equation induction and numerical cases, construct a fixed scalar predecessor term and prove $P0 = 0$ and $P(Sn) = n$. Existential coverage then gives $x = 0 \vee x = S(Px)$: in the successor case $x = Sy$, apply P to obtain $Px = y$ and substitute.
2. Construct truncated subtraction and a numerical characteristic of actual equality. Prove its reflection on the entire original domain.
3. Compile each fixed quantifier-free formula into its characteristic equation and recover every contextual quantifier-free induction instance.

4. With that induction available, construct pairing and projections by a bounded square-root search and compile every fixed general primitive-recursive description into a scalar iterator term.

No pairing or faithful equality detector is assumed to obtain predecessor. No appeal to standard-model correctness replaces an internal equality. Every induction fixes its passive parameters; diagonal substitution occurs only into a previously proved identity.

For a target recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

let π, L, H be the proved pairing and projections. The compiler uses

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$; the two projection identities then prove the target defining equations. The construction is external recursion on each finite PR description, not an internal quantification over arbitrary functions.

Theorem C.1 (Whole-domain bi-interpretability and equiconsistency)**.** *The ordinary realization T_I and full first-order iPRA are bi-interpretable. In both theories $x = y$ defines the isomorphism from the original domain to its double interpretation. The native Choice-DI calculus is therefore a faithful proof presentation of a theory bi-interpretable and equiconsistent with iPRA.*

Proof. Interpret an iterator in iPRA by primitive recursion that ignores its counter argument in the update. iPRA proves its equations, literal equation induction, coverage and separation. In the reverse direction use the scalar compiler. The predecessor and equality-reflection proofs supply the successor axioms and every quantifier-free induction instance. All ordinary logical rules are preserved.

For two iterators with the same seed and provably equal updates, equation induction on the equality of their values proves uniqueness at every counter, with all parameters fixed. The corresponding argument applies to ordinary primitive recursion. Induct externally on a finite description: its double translation satisfies its original defining equations, so internal uniqueness identifies the two functions. This also proves coherence when substitution changes the staged syntactic description of an update.

The domain and equality are unchanged; hence the identity graph is internally total, bijective and preserves every interpreted operation. The displayed ordinary meanings of the logical macros and the paired binder give both formula composites by external induction on finite formula syntax. Clauses are similarly equivalent to their conditional universal meanings. The explicit falsity bridge preserves contradiction in both directions. Effective translations of finite proofs give equiconsistency, without a claim that either theory proves its own consistency. □

D Internal scalar predecessor bootstrap

This scalar bootstrap is inherited from the supplied scalar development [7] and is reproduced to keep the scalar checker self-contained. It is not a new contribution of this paper. The proof uses a factor map fixing odd states and a saturated-iteration construction of predecessor.

The available arithmetic terms are 0, S, and

$$I(n, a; u.h), \qquad I(0, a; u.h) = a, \qquad I(Sn, a; u.h) = h[I(n, a; u.h)/u].$$

The update binds only u. All other variables in it are fixed numerical parameters. We use equality reasoning, the already reconstructed zero/successor cases, and induction on a single literal equation. Every induction below keeps its other parameters fixed. Predecessor, successor injectivity, pairing, and induction on arbitrary quantifier-free formulas are not assumptions of the argument.

D.1 Elementary terms and two finite cycles

Write $1 = S0$ and use larger numerals as successor abbreviations. Define

$$C(n, a, b) = I(n, a; u.b), \qquad Z(n) = C(n, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The update variable of C is absent from its body. Consequently

$$C(0, a, b) = a, \qquad C(Sn, a, b) = b.$$

Numerical cases show that $Z(n)$ is 0 or 1. The usual commutative-monoid equations for addition follow by literal equation induction. For example, $0 + b = b$ and $Sa + b = S(a + b)$ are proved by induction on b; these yield commutativity by a further equation induction, and associativity follows by induction on the last summand. We use these equations to reorder finite sums. The notation $2a$ means $a + a$ and $3a$ means $a + a + a$; no multiplication operation is needed for this bootstrap.

Put

$$N(n) = I(n, 0; u.Z(u)), \qquad j(u) = Z(u) + 2N(u), \qquad M(n) = I(n, 0; u.j(u)).$$

The range of N is contained in $\{0, 1\}$: its zero value is 0, and its successor value is a value of Z. This uses numerical cases on its counter, not induction on a disjunction. Hence

$$N(n) + Z(N(n)) = 1, \qquad Z(Z(N(n))) = N(n), \qquad N(n + 2) = N(n). \tag{2}$$

Here the last equation follows from two successor computations and the preceding equation. Literal-equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n). \tag{3}$$

For the first equation the successor step adds two to the argument of N; the second follows by one computation. In the third, increasing n adds three to the left argument and therefore applies Z once to both sides of the induction equation.

Finite computations now give

$$j(0) = 1, \qquad j(1) = 2, \qquad j(2) = 0.$$

The proof below needs just these three values and $M(Sn) = j(M(n))$; it does not assume a division algorithm or a theorem characterizing M as remainder modulo three.

D.2 A ramp with a global scaling identity

Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

The central invariant is

$$M(G(n)) = N(n). \tag{4}$$

Its zero case is immediate. For its successor step, retain the single induction equation and split the Boolean value $N(n)$. If it is 0, the next G state is $G(n)+1$; its M value is $j(0) = 1$, equal to $N(Sn)$. If it is 1, the next G state is $G(n) + 2$; two computations of M change its value from 1 to 2 and then to 0, again equal to $N(Sn)$. This proves the invariant by literal-equation induction.

It follows that

$$G(Sn) = G(n) + N(n) + 1, \qquad G(n + 2) = G(n) + 3.$$

Equation induction therefore yields

$$G(2n) = 3n. \tag{5}$$

A separate literal-equation induction proves

$$G(3n) = 3G(n) + N(n). \tag{6}$$

For completeness, its successor calculation is

$$\begin{aligned} G(3Sn) &= G(3n) + N(3n) + 4 = 3G(n) + 2N(n) + 4, \\ 3G(Sn) + N(Sn) &= 3G(n) + 3N(n) + 3 + Z(N(n)). \end{aligned}$$

The expressions agree in both Boolean cases for $N(n)$. The base is $0 = 0$. In particular, (6) was proved internally; it was not inferred from an intended floor-function interpretation of G.

Set

$$H(u) = C(N(u), G(u), u).$$

The crucial choice is to retain u when its parity is 1. The preceding equations give

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u). \tag{7}$$

To check the last identity, use $N(3u) = N(u)$ and split the Boolean $N(u)$. At 0, both sides are $3G(u)$ by (6); at 1, both are $3u$. This argument does not require that an arbitrary number has already been represented as $2v$ or $2v + 1$.

D.3 A comparator from scalar iteration

For every fixed update h, equation induction on n, keeping m, a and all parameters fixed, proves the semigroup identity

$$I(m + n, a; h) = I(n, I(m, a; h); h).$$

The base is the zero computation and the step applies h to the induction equation. This identity will also be used in the later pairing construction.

For a fixed update h, equation induction on y, with its initializer a and all parameters of h fixed, proves the rotation identity

$$I(Sy, a; h) = I(y, h(a); h). \tag{8}$$

Both sides have base $h(a)$, and their successor equations apply the same h to the previous equality. Here and below $I(y, a; h)$ is an abbreviation for one fixed finite scalar term, not quantification over functions.

By (7), another equation induction gives

$$I(y, 3a; H) = 3I(y, a; H). \tag{9}$$

Its base is $3a = 3a$. In the step the fixed-parameter induction equation rewrites the new left side as

$$H(3I(y, a; H)) = 3H(I(y, a; H)).$$

The initializer parameter stays a throughout this induction. Equation induction also gives $I(y, 1; H) = 1$, since $H(1) = 1$.

Define

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N\big(I(y, E(x); H)\big).$$

Then the following entire recursion specification is provable:

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y). \tag{10}$$

The first equation uses $E(0) = 1$ and the fixed-point identity for H. The second uses $E(Sx) = 2E(x)$ and $N(2a) = 0$. For the third,

$$\begin{aligned} L(Sx, Sy) &= N\big(I(Sy, 2E(x); H)\big) \\ &= N\big(I(y, H(2E(x)); H)\big) \\ &= N\big(I(y, 3E(x); H)\big) \\ &= N\big(3I(y, E(x); H)\big) \\ &= N\big(I(y, E(x); H)\big) = L(x, y). \end{aligned}$$

The successive steps use rotation, $H(2a) = 3a$, iterator scaling, and $N(3a) = N(a)$. Thus L has the comparison equations at all arguments internally. Its standard-model meaning as the indicator of $x \leq y$ is motivation, not a premise of the following argument.

D.4 Saturated iteration gives predecessor

We now need only the three equations (10). Define

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The subscript in B_m records a fixed parameter; B_m is an open term abbreviation, not a function variable.

Numerical cases on e prove the selector equation

$$C(e, Sa, Sb) = SC(e, a, b).$$

Equivalently one may use equation induction on e, whose induction hypothesis is unused. Therefore

$$\begin{aligned} B_{Sm}(Su) &= C(L(SSu, Sm), Su, SSu) \\ &= C(L(Su, m), Su, SSu) = SB_m(u), \qquad (11) \\ B_{Sm}(0) &= C(L(S0, Sm), 0, S0) = C(L(0, m), 0, S0) = S0. \qquad (12) \end{aligned}$$

Literal-equation induction on t, with m fixed, proves

$$W(St, Sm) = SW(t, m). \tag{13}$$

The base is (12). The successor step is

$$\begin{aligned} W(SSt, Sm) &= B_{Sm}(W(St, Sm)) \\ &= B_{Sm}(SW(t, m)) \\ &= SB_m(W(t, m)) = SW(St, m). \end{aligned}$$

Only the second line uses the current induction hypothesis, at the same fixed m.

Two subsequent equation inductions give

$$L(Sn, n) = 0, \qquad W(n, n) = n. \tag{14}$$

The first uses the comparator recurrence. The second uses (13) in its successor step. The substitution $t = m = n$ is made into that already proved universally parameterized equation; no parameter of an active induction hypothesis is changed. Consequently

$$W(Sn, n) = B_n(n) = C(L(Sn, n), n, Sn) = n. \tag{15}$$

Finally define the finite scalar term

$$P(m) = I\big(m, 0; u.C(L(SSu, m), u, Su)\big). \tag{16}$$

Its zero equation is immediate. To justify its successor equation without assuming a separate extensionality rule for iterator bodies, put

$$V(t, m) = I\big(t, 0; u.C(L(SSu, Sm), u, Su)\big).$$

The comparator recurrence identifies this update pointwise with $B_m(u)$. Equation induction on t, with m fixed, gives $V(t, m) = W(t, m)$: their base values are 0, and the successor step uses the pointwise update equation and then the induction equation. Now (15) yields

$$\boxed{P(0) = 0, \qquad P(Sm) = V(Sm, m) = W(Sm, m) = m.}$$

D.5 Dependency boundary

All terms in this section are built from $0, S, I$ by finite substitution. All inductions have a single equation as their invariant. Numerical cases and equality transport are the already reconstructed logical resources. Successor injectivity may now be obtained by applying P to an assumed equation $Sx = Sy$. Equality tests, reflection of quantifier-free formulas, pairing, and the recovery of general primitive recursion can then be developed in the established order. They were not used in order to manufacture predecessor.

This argument is a written internal derivation in the stated theory. The finite implementation checks do not constitute a proof-assistant formalization of this entire metatheorem.

E From literal-equation induction to full intuitionistic PRA

This section supplies an unconditional completion theorem. It is adapted with attribution from the current forward-left draft [8]. In the scalar application used here, every function occurring in the proof is already a scalar term: predecessor is supplied by Appendix D, the selector is $C(n, a, b) = I(n, a; u.b)$, addition is $x + y = I(y, x; u.Su)$, and truncated subtraction is $d(x, y) = I(y, x; u.Pu)$. All other functions below are finite compositions of these. Thus this proof does not presuppose that arbitrary primitive recursion has already been compiled. The QFI conclusion first applies to the scalar language; the general PR compiler comes afterward. Literal-equation induction, together with the primitive-recursive defining equations, predecessor coverage, and separation of 0 and 1, proves induction for every quantifier-free formula. In particular, the proof does not assume a faithful numerical equality test, stability or decidability of equality, successor uniqueness, or any commutation law for an iterator. We construct the equality test and prove its reflection property for arbitrary elements of the numerical domain.

E.1 The exact hypotheses

Work in ordinary intuitionistic first-order logic with equality and genuine falsity $\bot$. The language has one numerical sort and a symbol for each fixed, finite primitive-recursive description. Distinct descriptions are not identified merely because they compute the same standard function. Let PRE contain the defining equations for zero, successor, projections, composition, and primitive recursion. In particular, choose descriptions P and C with equations

$$P(0) = 0, \qquad P(Sx) = x, \tag{17}$$

$$C(0, a, b) = a, \qquad C(Sx, a, b) = b. \tag{18}$$

Write $1 = S0$. In addition to PRE, assume

$$\mathsf{Cov}: \quad \forall x \, (x = 0 \vee x = S(Px)), \tag{19}$$

$$\mathsf{Sep}: \quad 0 = 1 \longrightarrow \bot, \tag{20}$$

and induction for each *literal equation* $E(n, \vec{p})$:

$$\forall \vec{p} \, \left[(E(0, \vec{p}) \wedge \forall n (E(n, \vec{p}) \longrightarrow E(Sn, \vec{p}))) \longrightarrow \forall n \, E(n, \vec{p}) \right]. \tag{21}$$

Here the schematic expression E is a single equality of numerical terms; the displayed universal quantifiers range only over numbers. Every description and every instance of the schema is finite external syntax.

The axiom formulation (21) and the usual contextual induction rule are interchangeable. For example, if

$$\Gamma \vdash E(0, \vec{p}), \qquad \Gamma, E(z, \vec{p}) \vdash E(Sz, \vec{p}),$$

where z is absent from Γ and the passive parameters, discharge the second premise's equation and universally introduce z. The axiom then gives $\Gamma \vdash \forall n E(n, \vec{p})$, which can be instantiated at any target term. Conversely, assume the antecedent of the axiom and apply the contextual rule with fresh variables. This uses the ordinary intuitionistic logical rules and permits arbitrary, including quantified, formulas in Γ.

E.2 Constructor separation and two elementary tests

Define the sign description and the countdown description by

$$\mathrm{sg}(u) = C(u, 0, 1), \qquad K(u) = C(u, 1, Pu). \tag{22}$$

Their defining equations imply

$$\mathrm{sg}(0) = 0, \qquad \mathrm{sg}(Sx) = 1, \tag{23}$$
$$K(0) = 1, \qquad K(Sx) = x. \tag{24}$$

Constructor laws. Successor is injective: applying P to $Sx = Sy$ gives $x = y$ by (17). Successor never equals zero. Indeed, $Sx = 0$ gives $1 = 0$ by applying sg and using (23); symmetry and (20) give $\bot$. Consequently $1 \neq 0$, where $A \neq B$ abbreviates $A = B \to \bot$. None of these arguments uses induction.

The sign test. For every u,

$$\mathrm{sg}(u) = 0 \vee \mathrm{sg}(u) = 1, \tag{25}$$
$$\mathrm{sg}(u) = 0 \quad \longleftrightarrow \quad u = 0, \tag{26}$$
$$\mathrm{sg}(u) = 1 \quad \longleftrightarrow \quad u \neq 0. \tag{27}$$

To prove the range assertion, split (19). The branch $u = 0$ gives sign 0, and the branch $u = S(Pu)$ gives sign 1. The implication $u = 0 \to \mathrm{sg}(u) = 0$ is computation. For its converse, the successor branch of coverage would make $\mathrm{sg}(u) = 0$ imply $1 = 0$, a contradiction; in that branch explosion gives $u = 0$ as well. Similarly, if $\mathrm{sg}(u) = 1$ and $u = 0$, computation gives $0 = 1$. Conversely, $u \neq 0$ excludes the zero branch of coverage, and the successor branch computes the sign as 1.

Coverage and (18) also give, for all u, v,

$$C(u, v, v) = v. \tag{28}$$

Both coverage branches compute to v, so this identity again needs no induction.

The countdown test and the notation W. For every u,

$$K(u) = 0 \quad \longleftrightarrow \quad u = 1. \tag{29}$$

If $u = 1 = S0$, (24) gives $K(u) = 0$. Conversely, assume $K(u) = 0$ and split coverage. If $u = 0$, then $K(u) = 1$, contradicting separation, and explosion gives $u = 1$. If $u = S(Pu)$, computation gives $K(u) = Pu$; hence $Pu = 0$ and $u = S0 = 1$.

If a presentation uses a description named W with defining equations

$$W(0) = 1, \qquad W(Sx) = x,$$

then it proves $K(u) = W(u)$ for every u already from coverage. In the zero branch both sides are 1; in the branch $u = S(Pu)$ both sides are Pu. Thus K and W are provably equal on the whole domain, even if they are distinct descriptions. This equality does not appeal to standard-model agreement or to uniqueness of an arbitrary recurrence.

E.3 Four elementary uses of equation induction

Choose descriptions for addition and truncated subtraction, with recursion on the second argument:

$$x + 0 = x, \qquad x + Sy = S(x + y), \tag{30}$$
$$d(x, 0) = x, \qquad d(x, Sy) = P(d(x, y)). \tag{31}$$

Using these equations and (21), we now derive

$$0 + y = y, \tag{32}$$
$$(Sx) + y = S(x + y), \tag{33}$$
$$d(Sx, Sy) = d(x, y), \tag{34}$$
$$d(Sa, a) = 1. \tag{35}$$

Each induction below has a single equation as its induction formula.

For (32), induct on y. The base is $0 + 0 = 0$. The step is $0 + Sy = S(0 + y) = Sy$.

For (33), hold x fixed and induct on y. At zero both sides are Sx. At a successor the induction hypothesis gives

$$(Sx) + Sy = S\big((Sx) + y\big) = S\big(S(x + y)\big) = S(x + Sy).$$

Combining (33) with the defining equation for addition gives the useful rearrangement

$$(Sy) + r = y + Sr. \tag{36}$$

Indeed, both sides are $S(y + r)$.

For (34), hold x fixed and induct on y. The base computes as

$$d(Sx, S0) = P(d(Sx, 0)) = P(Sx) = x = d(x, 0).$$

Applying P to the induction equation gives the step:

$$d(Sx, S(Sy)) = P(d(Sx, Sy)) = P(d(x, y)) = d(x, Sy).$$

Finally, induct on a for (35). The base is $d(S0, 0) = S0 = 1$. At a successor,

$$d(S(Sa), Sa) = d(Sa, a) = 1,$$

using (34) and the induction hypothesis.

E.4 An equation invariant that reflects actual equality

The remaining arithmetic ingredient is the following equation, valid for all x, y:

$$C\big(\mathrm{sg}(d(x, y)),\ x,\ y + d(x, y)\big) = x. \tag{37}$$

Its meaning is that a nonzero subtraction residual reconstructs the original argument by addition. The selector expresses this assertion as one literal equation, so equation induction applies directly.

Hold x fixed and induct on y. At $y = 0$, subtraction gives residual x, and (32) gives $0 + x = x$. The desired equation is consequently $C(\mathrm{sg}(x), x, x) = x$, which is (28).

For the step, assume (37) at y and abbreviate

$$r = d(x, y), \qquad r' = Pr = d(x, Sy).$$

These are local term abbreviations, not additional function symbols. Split coverage for r'.

If $r' = 0$, then $\mathrm{sg}(r') = 0$, and the successor instance of (37) reduces immediately to $x = x$.

In the other branch, $r' = S(Pr')$, so constructor separation gives $r' \neq 0$. Split coverage for r. Its zero branch would imply $r' = P0 = 0$, a contradiction, from which the desired step equation follows. In its successor branch,

$$r = S(Pr) = Sr', \qquad \mathrm{sg}(r) = 1.$$

The induction equation therefore computes to $y + r = x$. Meanwhile, (36) gives

$$(Sy) + r' = y + Sr' = y + r = x.$$

Since r' is a successor, $\mathrm{sg}(r') = 1$, so this is exactly the branch selected in the successor instance of (37). This completes the equation induction. All case distinctions came from coverage; no induction on an implication or a disjunction was used.

Reflection at residual 1. We have, for arbitrary a, b,

$$d(Sa, b) = 1 \quad \longleftrightarrow \quad a = b. \tag{38}$$

For the forward direction, substitute $x = Sa$, $y = b$ into (37). If $d(Sa, b) = 1$, the sign is 1 and the selector returns its second branch, giving

$$b + 1 = Sa.$$

The defining equation for addition gives $b + 1 = S(b + 0) = Sb$. Thus $Sb = Sa$, and applying P gives $b = a$, hence $a = b$. Conversely, $a = b$ and (35) give $d(Sa, b) = d(Sa, a) = 1$.

Define the equality characteristic by the fixed description

$$e(a, b) = \mathrm{sg}\big(K(d(Sa, b))\big). \tag{39}$$

The sign, countdown, and residual reflection lemmas give

$$e(a, b) = 0 \quad \longleftrightarrow \quad a = b, \tag{40}$$

$$e(a, b) = 0 \ \vee \ e(a, b) = 1. \tag{41}$$

In detail, $e(a, b) = 0$ is equivalent to $K(d(Sa, b)) = 0$ by (26), which is equivalent to $d(Sa, b) = 1$ by (29), and then to $a = b$ by (38). The range assertion is (25) applied to $K(d(Sa, b))$.

The reflection property concerns the logic's actual equality throughout the domain. It was not assumed as part of a numerical comparison symbol, and its proof did not restrict a or b to standard numerals. It also yields decidable equality. In the $e(a, b) = 0$ branch, (40) gives $a = b$. In the $e(a, b) = 1$ branch, an assumption $a = b$ would give $e(a, b) = 0$ and therefore $0 = 1$, contradicting separation. Hence

$$\forall a \, \forall b \, \big(a = b \vee (a = b \to \bot)\big) \tag{42}$$

is a theorem, not an extra hypothesis.

E.5 Compilation of all quantifier-free formulas

For each fixed quantifier-free formula $A(\vec{p})$, construct a term $b_A(\vec{p})$ by external recursion on its finite syntax. Take equality, $\bot$, $\wedge$, $\vee$, and $\to$ as the primitive formula constructors; negation is implication into $\bot$. Set

$$b_{s=t} = e(s,t), \qquad b_\bot = 1, \tag{43}$$
$$b_{A\wedge D} = C(b_A, b_D, 1), \tag{44}$$
$$b_{A\vee D} = C(b_A, 0, b_D), \tag{45}$$
$$b_{A\to D} = C(b_A, b_D, 0). \tag{46}$$

Every term here is a finite composition of fixed primitive-recursive descriptions; if desired, projections and composition give it a single description symbol. No internal quantification over formulas or programs is involved.

An external induction on the syntax of A proves, within the arithmetic theory,

$$(b_A = 0 \vee b_A = 1) \quad \text{and} \quad (b_A = 0 \longleftrightarrow A). \tag{47}$$

Here are the full connective checks. In every case $b_A = 1$ implies $\neg A$: if A held, the already established implication $A \to b_A = 0$ would contradict separation.

Atomic formulas and falsity. For an equation, both claims are (40)–(41). For falsity the range assertion is $1 = 0 \vee 1 = 1$, with its right disjunct proved by reflexivity. Also $1 = 0 \to \bot$ follows from separation and symmetry, while $\bot \to 1 = 0$ follows by explosion. Thus $1 = 0$ is equivalent to $\bot$, as required.

Conjunction. Split $b_A = 0 \vee b_A = 1$. In the zero branch, A holds and $b_{A\wedge D} = b_D$. Its range follows from the range of b_D, and its being zero is equivalent to D, hence to $A \wedge D$ under the known A. In the one branch, $\neg A$ holds and $b_{A\wedge D} = 1$. Its range is immediate. If it were zero, separation would give $\bot$, hence $A \wedge D$; conversely an assumption $A \wedge D$ contradicts $\neg A$, and explosion gives $b_{A\wedge D} = 0$.

Disjunction. In the branch $b_A = 0$, the formula A holds and $b_{A\vee D} = 0$. The range is immediate, and both $b_{A\vee D} = 0$ and $A \vee D$ hold, proving their equivalence. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\vee D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D and hence $A \vee D$. Conversely, from $A \vee D$, the A alternative contradicts $\neg A$ and yields $b_D = 0$ by explosion, while the D alternative yields $b_D = 0$ directly. This proves the required equivalence in both alternatives.

Implication. In the branch $b_A = 0$, the formula A holds and $b_{A\to D} = b_D$. Its range follows from the range of b_D. If its value is zero, then D, which gives $A \to D$. Conversely, $A \to D$ and the known A give D, hence $b_D = 0$. In the branch $b_A = 1$, we have $\neg A$ and $b_{A\to D} = 0$. Its range is immediate. To prove $A \to D$, assume A; this contradicts $\neg A$, and explosion gives D. Thus both $b_{A\to D} = 0$ and $A \to D$ hold in this branch, and their equivalence follows. This argument uses the proved two-value range of the finite characteristic. It does not use excluded middle for arbitrary formulas or change the surrounding intuitionistic logic.

This completes the external syntactic induction proving (47) for every quantifier-free formula.

E.6 Recovery of induction and the full theory

Let $A(n, \vec{p})$ be any fixed quantifier-free formula. Suppose an arbitrary retained context Γ proves its base and its induction step:

$$\Gamma \vdash A(0, \vec{p}), \qquad \Gamma, A(z, \vec{p}) \vdash A(Sz, \vec{p}),$$

with the usual freshness condition on z. By (47), the base gives $\Gamma \vdash b_A(0, \vec{p}) = 0$. Under the additional assumption $b_A(z, \vec{p}) = 0$, the same equivalence gives $A(z, \vec{p})$, the given step gives $A(Sz, \vec{p})$, and the equivalence gives $b_A(Sz, \vec{p}) = 0$. Apply literal-equation induction to $b_A(n, \vec{p}) = 0$ and then apply its reflection implication. The result is $\Gamma \vdash A(t, \vec{p})$ for every target term t. Thus the theory proves the full contextual quantifier-free induction rule, including in arbitrary quantified contexts.

Completion theorem. In the common language of fixed primitive-recursive descriptions, the theory

$$\mathsf{PRE} + \mathsf{Cov} + \mathsf{Sep} + \mathsf{EI}$$

has exactly the same sequents as ordinary first-order intuitionistic primitive recursive arithmetic with all quantifier-free induction instances. The forward simulation follows from the recovered quantifier-free induction, successor injectivity, and successor separation; all primitive-recursive defining equations and all logical rules are shared. For the reverse simulation, literal-equation induction is an instance of quantifier-free induction, and successor separation gives Sep. Quantifier-free induction proves Cov: at zero choose its left disjunct, and at a successor use $Sx = S(P(Sx))$, which follows from $P(Sx) = x$, for its right disjunct.

Both interpretations therefore use the full domain, defined by $x = x$, and preserve every function symbol, equality, logical connective, and quantifier. Both composites have the identity isomorphism, with graph $x = y$. The theories are consequently identity-domain bi-interpretable and equiconsistent. This completion theorem is unconditional once the literal-equation induction rule has been obtained; it has no dependency on a classical successor-uniqueness reduction.

F Pairing, primitive recursion, and internal numerical composites

This appendix is adapted with attribution from the progress-closure arithmetic development [9]. Its prerequisites have now been proved: scalar predecessor, arithmetic reflection, and full quantifier-free induction. Pairing is a derived proof tool; it is not part of any checker.

F.1 Pairing from a bounded square-root search

Once QFI is available, a familiar square-based injection gives a particularly direct compiler. Write $a \le b$ for $d(a,b) = 0$. The usual linear-order and semiring identities follow by fixed quantifier-free inductions from the successor, addition, and monus equations. Multiplication itself is the scalar term

$$a \cdot b = I(b, 0; u.u + a).$$

Only primitive-recursive arithmetic is involved in the following bounded root search; there is no minimization over an unbounded domain.

For a fixed parameter n, put

$$U_n(v) = C(d((Sv)^2, n), Sv, v), \qquad R_n(i) = I(i, 0; v.U_n(v)), \qquad q(n) = R_n(n).$$

The update increments v exactly when $(v+1)^2 \le n$, and otherwise retains it. QFI in i, with n fixed, proves

$$R_n(i) \le i, \qquad R_n(i)^2 \le n, \qquad R_n(i) = i \ \vee \ n < (S(R_n(i)))^2. \tag{Root invariant}$$

The base has $R_n(0) = 0$. If the update increments, its test contradicts the strict-upper-bound alternative in the third invariant. Thus the old state equals i, and the new state equals Si; its square bound is the increment test. If the update does not increment, the strict upper square bound holds and the state remains fixed. An already established strict upper square bound forces every subsequent update to remain fixed. These are exactly the cases needed for the three displayed invariants.

At $i = n$, either the strict upper square bound already holds or $R_n(n) = n$, in which case $n < (n+1)^2$ supplies it. Thus

$$q(n)^2 \le n < (q(n)+1)^2. \tag{Root}$$

Monotonicity of squares shows uniqueness of a number satisfying Root. All formulas just used are quantifier free in the scalar language, with fixed n during the induction.

Now define

$$\pi(a,b) = (a+b)^2 + a, \qquad L(n) = d(n, q(n)^2), \qquad H(n) = d(q(n), L(n)).$$

For $k = a + b$, one has $k^2 \le k^2 + a < (k+1)^2$ since $a \le k$. Root uniqueness gives $q(\pi(a,b)) = a + b$, and therefore

$$L(\pi(a,b)) = a, \qquad H(\pi(a,b)) = b.$$

Surjectivity of π is unnecessary. The original numerical domain is never replaced by the range of this coding; pairing is used only to store the finite state of a fixed primitive recursion. This standard square pairing is a convenient reconstruction here, not a claimed new arithmetic basis.

F.2 Compiling each primitive recursion

Suppose the fixed target description is

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}).$$

Assume by external structural induction that the earlier g, h already have scalar translations g^*, h^*. Define

$$W(u,\vec{p}) = \pi(SL(u), h^*(L(u), H(u),\vec{p})),$$

$$V(n,\vec{p}) = I(n, \pi(0, g^*(\vec{p})); u.W(u,\vec{p})), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

The update has only its state variable bound, with every p_i fixed. Equation induction proves $L(V(n,\vec{p})) = n$. The two projection equations and scalar computation then prove both defining equations for f^* internally. Initial functions and composition translate directly. This completes the translation of every fixed finite PR description without adding a universal evaluator.

F.3 Why both numerical round trips are internal

Two fixed scalar iterators with the same seed and provably equal updates are internally equal at all counters: use literal-equation induction comparing their values. Their base agrees, and their successor equations plus congruence prove the step. The same uniqueness argument works for ordinary primitive recursion.

For an ordinary PR description, induct externally on its finite construction. The double translation satisfies its original defining equations, so internal equation induction identifies it with the original operation. For a native scalar description, earlier translated updates agree by the external induction hypothesis; scalar uniqueness identifies the double-translated iterator. These proofs hold at arbitrary internal counters.

Thus the domain relation $x = y$ is provably a total bijection preserving every fixed numerical operation in both composites. The formula constructions and round trips in Section B give the corresponding formula composites. Both interpretations preserve falsity, taking native $0 = 1$ to an ordinary contradiction and ordinary $\bot$ to native $0 = 1$. Finite proof translations therefore establish equiconsistency.

References

[1] ProveIt contributors. *Concise presentations of intuitionistic primitive recursive arithmetic*, README at commit `443cff49563457a48ee0f663012d56c4bdb58fa0`. https://github.com/VladimirReshetnikov/ProveIt/tree/443cff49563457a48ee0f663012d56c4bdb58fa0/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA. All repository references below use this exact commit.

[2] ProveIt contributors. *Uniform Step*, `Research/uniform_step/`: report, claims, complete checker, and fixtures. The frozen matched source is `code/checker.py`.

[3] ProveIt contributors. *Canonical recursors* and *Checked refinement*, `Research/canonical_recursors/` and `Research/checked_refinement/`. Compact source-size comparisons and their documented input contracts.

[4] ProveIt contributors. `Lean/ConciseIPRA/NativeZeroStep.lean`. Declarations `NSPrf`, `nsprf_iff_nprf`, `nsprf_iff_pra`, and `NSPrf.normal`.

[5] ProveIt contributors. `Lean/ConciseIPRA/BiInterpretation.lean`. Declarations `biInterp`, `teq_roundTrip`, `pra_roundTrip`, and `teq_consistent_iff`.

[6] ProveIt contributors. `Lean/ConciseIPRA/NativeBiInterpretation.lean`. Declarations `native_iff_pra`, `pra_iff_native`, `native_roundTrip`, `pra_native_roundTrip`, and `native_inconsistent_iff`.

[7] ProveIt contributors. Inherited scalar predecessor bootstrap, `Research/certified_zero/reference/scalar_predecessor.tex`. Reproduced in the inherited appendix.

[8] ProveIt contributors. Inherited arithmetic reflection, `Research/certified_zero/reference/equality_reflection.tex`. Reproduced in the inherited appendix.

[9] ProveIt contributors. Inherited primitive-recursion compiler, `Research/certified_zero/reference/primitive_recursion.tex`. Reproduced in the inherited appendix, together with `inherited_choice_basis.tex` and `inherited_ordinary_bridge.tex`.

[10] Saeed Salehi. Decidable formulas of intuitionistic primitive recursive arithmetic. *Reports on Mathematical Logic* **36** (2002), 55–61. https://rml.tcs.uj.edu.pl/rml-36/36-SALEHI.pdf.